Application of Quantum Extreme Learning Machines for QoS Prediction of Elevators' Software in an Industrial Context

Xinyi Wang
xinyi@simula.no
Simula Research Laboratory and University of Oslo
Oslo, Norway

Shaukat Ali
shaukat@simula.no
Simula Research Laboratory and Oslo Metropolitan University
Oslo, Norway

Aitor Arrieta
aarrieta@mondragon.edu
Mondragon University
Mondragon, Spain

Paolo Arcaini
arcaini@nii.ac.jp
National Institute of Informatics
Tokyo, Japan

Maite Arratibel
marratibel@orona-group.com
Orona
San Sebastian, Spain

ABSTRACT

Quantum Extreme Learning Machine (QELM) is an emerging technique that utilizes quantum dynamics and an easy-training strategy to solve problems such as classification and regression efficiently. Although QELM has many potential benefits, its real-world applications remain limited. To this end, we present QELM's industrial application in the context of elevators, by proposing an approach called QUELL. In QUELL, we use QELM for the waiting time prediction related to the scheduling software of elevators, with applications for software regression testing, elevator digital twins, and real-time performance prediction. The scheduling software is a classical software implemented by our industrial partner Orona, a globally recognized leader in elevator technology. We assess the performance of QUELL with four days of operational data of a real elevator installation with various feature sets and demonstrate that QUELL can efficiently predict waiting times, with prediction quality significantly better than that of classical ML models employed in a state-of-the-practice approach. Moreover, we show that the prediction quality of QUELL does not degrade when using fewer features. Based on our industrial application, we further provide insights into using QELM in other applications in Orona, and discuss how QELM could be applied to other industrial applications.

CCS CONCEPTS

• **Software and its engineering**; • **Computer systems organization** → **Quantum computing**;

KEYWORDS

Quantum computing, quantum extreme learning machines, quantum reservoir computing, industrial elevators, regression testing

FSE Companion '24, July 15–19, 2024, Porto de Galinhas, Brazil

ACM ISBN 979-8-4007-0658-5/24/07
https://doi.org/10.1145/3663529.3663859

ACM Reference Format:
Xinyi Wang, Shaukat Ali, Aitor Arrieta, Paolo Arcaini, and Maite Arratibel. 2024. Application of Quantum Extreme Learning Machines for QoS Prediction of Elevators' Software in an Industrial Context. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion '24), July 15–19, 2024, Porto de Galinhas, Brazil.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3663529.3663859

1 INTRODUCTION

Orona is a Spanish company well-renowned for building elevators [39], i.e., vertical transportation systems responsible for safely, comfortably, and efficiently transporting passengers across different floors. At the core of elevators, there is the *dispatcher*, a scheduling software that assigns an elevator to each call by maintaining an acceptable *quality of service* (QoS). A commonly used metric for measuring the QoS of the elevators is the *Average Waiting Time* (AWT), i.e., the average time passengers have to wait, from when they press the call button until the time the elevator arrives. AWT is widely used to test dispatchers under various building configurations to identify quality issues [8, 53], followed by a proper dispatcher configuration to achieve acceptable performance.

The dispatcher software undergoes evolution like any other software system, requiring regression testing. In Orona, a regression test oracle is employed during design time in both Software in the Loop (SiL) and Hardware in the Loop (HiL) settings. To reduce testing costs, such regression test oracle was proposed to be replaced by a machine learning(ML)-based test oracle [19].

Currently, as the DevOps paradigm emerges in the context of Cyber-Physical Systems (CPSs), Orona is interested in deploying such oracles also in operation to see whether the QoS of the elevators meets acceptable values. However, applying existing ML-based oracles is not straightforward and poses challenges. Indeed, during design time, several features are available. For example, the passengers' weight is a feature that an ML-based test oracle can use at design time to determine whether the AWT of passengers is within an acceptable threshold in a given time window. However, some of these features, such as the passengers' weights, are not easy to measure at operation time; hence, they cannot be used as features for ML models to be used at operation time. Moreover, elevators must be configured according to the constraints imposed by the building where they are installed. For example, in some elevators, it is possible to extract the exact distance traveled by each passenger.

In contrast, in other elevators, it is not possible. As a result, different number of features are available for different elevator installations.

Hence, in Orona, there is an increasing need for ML-based test oracles that can be trained on a variable number of features, ranging from a small number to a large number of features, depending on the installation configurations. Moreover, there is interest in using ML models at the operation time to support analyses such as the predictions of various QoS, runtime verification and monitoring, and advanced analyses with digital twins of elevators [56].

To cater to the above needs of Orona, we explore the use of Quantum Extreme Learning Machine (QELM) [36], by proposing a QELM-based approach called *QUantum Extreme Learning eLevator* (QUELL). QELM is a quantum machine learning technique that uses quantum dynamics of quantum reservoirs to enable a simple machine learning model (e.g., a linear regression model) to be efficiently trained with a limited number of features, but still provide good prediction quality. The ability of the quantum reservoir to map input data with limited dimensions into a higher-dimensional quantum space contributes to this efficiency. QUELL uses this ability to map a limited number of features into higher-dimensional information and enables robust prediction capabilities.

Our contributions are: (1) we present the first QELM application to an industrial prediction task with real industrial data; (2) we assess the effectiveness of QUELL with four days of operational data of a real elevator installation with various feature sets; (3) we compare QUELL with the existing ML-based regression models employed in the same industrial context for predicting AWT. The results showed that QUELL significantly outperforms the classical regression models for the prediction task, thereby demonstrating QELM's potential benefits in the industrial context; (4) we discuss the application of QUELL to various contexts at Orona based on our results, followed by a discussion on the potential applications in other industrial contexts; (5) we present open research questions related to QELM applications in software engineering that are valuable for software engineering researchers and practitioners.

2 BACKGROUND

We provide quantum computing basics and background on QELM.

Quantum Computing Basics. Quantum computers compute with *quantum bits* (i.e., *qubits*). Unlike a classical bit taking values in $\{0, 1\}$, a qubit can be in *superposition*, i.e., in states 0 and 1 simultaneously, with specific probabilities of collapsing into either 0 or 1 upon measurement. A *quantum state* can be represented as a *state vector*, e.g., a one-qubit state can be represented as $|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle$, where α_0 and α_1 are the *amplitude* of the quantum state. $|\alpha_0|^2$ and $|\alpha_1|^2$ represent the probability of being in state $|0\rangle$ and $|1\rangle$ when observed. Similarly, a D-qubit state vector can be represented as:

$$|\psi\rangle = \begin{pmatrix} \alpha_0 \\ \vdots \\ \alpha_d \end{pmatrix} = \alpha_0 |0\rangle + \ldots + \alpha_d |d\rangle \qquad \text{with } d = 2^D$$

$|\psi\rangle$ is a normalized d-dimensional vertical vector, i.e., $\sum_{i=0}^{d} |\alpha_i| = 1$.

Quantum computers are currently programmed with *quantum circuits*. A quantum circuit consists of a series of *quantum gates* (e.g., a Hadamard gate that places a qubit into a superposition) performing computations on qubits, i.e., evolving the quantum state over time, guided by the principles of quantum mechanics implemented in the quantum gates. This time evolution process is defined by a *Hamiltonian*[1] H applied to the quantum circuit. We represent the process over a period of time Δt as $|\psi\rangle = exp(-iH\Delta t\,|\psi_0\rangle)$, where the quantum state evolves from $|\psi_0\rangle$ to $|\psi\rangle$. In this equation, $exp(-iH\Delta t)$ is a quantum gate, i.e., a unitary operator. The operator can also be represented as U and the computation process as $|\psi\rangle = U\,|\psi_0\rangle$. Table 1 describes the gates used in QUELL.

Table 1: Descriptions of quantum gates used in QUELL

Gate	Description
Pauli-Z (Z/σ_z)	It rotates a qubit around the z-axis with π radians.
Pauli-X (X/σ_x)	It rotates a qubit around the x-axis with π radians.
Pauli-Y (Y/σ_y)	It rotates a qubit around the y-axis with π radians.
$RX(\theta)$	It rotates a qubit around the x-axis with θ radians.
$RY(\theta)$	It rotates a qubit around the y-axis with θ radians.
$RZ(\theta)$	It rotates a qubit around the z-axis with θ radians.
Controlled-NOT (CNOT/CX)	A two-qubit gate with a control and a target qubit. If the control qubit is $\|1\rangle$, the target qubit rotates around the x-axis with π radians.
Controlled-Z (CZ)	A two-qubit gate with a control and a target qubit. If the control qubit is $\|1\rangle$, the target qubit rotates around the x-axis with π radians.

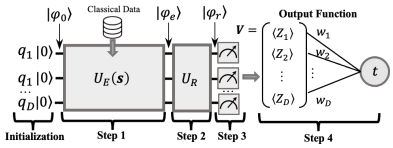


Figure 1: Implementation of QELM

At the end of circuit execution, the final quantum state is measured with a set of *observables* (unitary operators) as the output of the circuit. *Pauli operators* (i.e., X, Y, Z) are usually used to obtain partial information of x, y, or z basis from each qubit. The measured result of each qubit is determined by the expectation values. For example, if the j-th qubit in z basis is measured, the observed expectation value of the operator Z is represented as $\langle Z^j \rangle$.

Quantum Extreme Learning Machine. *Extreme learning machine* (ELM) is a feedforward neural network that uses fixed, nonlinear dynamics to extract information from inputs efficiently. It has neurons in the hidden layers and an output function [25]. The parameters of hidden layers are fixed and assigned randomly instead of being optimized during training, as in classic neural networks. ELM only needs to train the weights of the output function, which is usually a linear regression algorithm to compute the output weights.

A *quantum extreme learning machine* (QELM) is the quantum counterpart of an ELM. A key advantage of QELMs is that they can use the complex dynamics of a quantum system evolution process in an exponentially large quantum space. They enable intricate feature mapping for classical data inputs. The quantum circuit providing high dynamics is called *quantum reservoir*. The rich dynamics allow the use of far fewer resources than classical ELM, which instead requires extensive hidden layers [16, 36]. The QELM's implementation has four steps (see Fig. 1).

Step 1: During the training process, a classical input vector $\boldsymbol{s}$ is first given as input to a *quantum encoder* circuit to be transformed

[1] A Hamiltonian defines the total energy of a quantum system. It is specified as a *hermitian matrix*.

into quantum states. The quantum encoder is a set of gates containing parameterized quantum gates. Suppose the quantum circuit is initialized as $|\psi_0\rangle$. The quantum encoder with the classical input is applied on the circuit, obtaining the state $|\psi_e\rangle = U_E(\boldsymbol{s})\,|\psi_0\rangle$, where U_E represents the unitary of the encoder.

Step 2: Then, the reservoir is applied to the quantum system. The output state of the encoder goes into a quantum reservoir circuit. The parameters of the quantum reservoir circuits are fixed and randomly assigned. U_R is the unitary of the reservoir. As output, it produces the state $|\psi_r\rangle = U_R U_E(\boldsymbol{s})\,|\psi_0\rangle$.

Step 3: Then, a set of observables are applied on the final state $|\psi_r\rangle$ to obtain output information from the quantum circuit. Suppose that there are D qubits and the Pauli Z observable is applied on each qubit to extract information in z basis. The output vector of measured values is $\boldsymbol{V} = [\langle Z^1\rangle, \ldots, \langle Z^D\rangle]$.

Step 4: Finally, as a non-linear function is used in the reservoir, linear regression is used to train the weights of the output function, mapping the observed values $\boldsymbol{V}$ to the target values t. The linear regression model tries to find the closest predicted target value t^{pre}:

$$t^{pre} = \sum_{k=1}^{M} w_k \left\langle Z^k \right\rangle$$

w_i are the weights of the output function. The training minimizes the deviation of the predicted value t^{pre} from the target value t.

3 INDUSTRIAL CONTEXT

With over 250,000 elevator installations worldwide, Orona is one of the largest elevator companies in Europe. A system of elevators aims to transport passengers between floors of a building as safely as possible while minimizing the time they need to wait for the elevator. One way to do so is by serving the calls quickly and taking the passengers to their destination in the minimum time possible. There are different ways to accomplish that. For example, faster engines can move elevators faster and, so, serve the calls quicker. Another way this paper considers is scheduling the elevators as optimally as possible. The elevator *dispatching algorithm* does this.

The dispatching algorithm is the key component responsible for optimally assigning an elevator to each call. This decision is made by considering inputs such as the current position of each elevator, its moving direction, and the number of stops it has already made. By analyzing these factors, the dispatching algorithm minimizes aspects like the passengers' waiting time or energy consumption.

Orona has an extensive suite of dispatching algorithms. Like any software system, a dispatching algorithm undergoes regular maintenance and evolution to address hardware obsolescence, adapt to legislative changes, incorporate new functionalities, and resolve bugs. *Regression testing* is the predominant technique to ensure a high code quality in response to this evolution. To test such systems, Orona employs simulation-based techniques at different levels. The first level refers to the Software in the Loop (SiL) test level. At this level, the domain-specific simulator ELEVATE [15] is employed. ELEVATE takes as inputs a *passenger file* and the *building configuration file*. The *passenger file* contains a set of passengers traveling through a building. Each passenger has different attributes, such as the floor they are arriving on, their destination floor, when they arrive, their weight, etc. The *building configuration file* contains aspects like the number of elevators, their speed, the number of each floor, etc. At this level, two types of tests are executed: (i) *short scenario tests* and (ii) *full-day tests*. Short scenario tests focus on examining specific functional properties in isolation. The anticipated results of these tests are achieved through the implementation of assertions, metamorphic testing [8], or manual methods. During the full-day tests, scenarios resembling a typical full-day or its sub-scenarios in the life-cycle of the elevator system are carried out. The anticipated results of these tests are linked to specific Quality of Service (QoS) values over time, including the AWT, which are obtained by re-running the test using a different algorithm or an older version. After the SiL phase, during the Hardware-in-the-Loop (HiL) testing phase, similar test processes are iterated, incorporating actual hardware components such as target processors, communication systems, real-time operating systems, and human-machine interfaces. The test cases executed at this level remain consistent with those in earlier stages, but the crucial distinction lies in real-time execution, making it a more costly process.

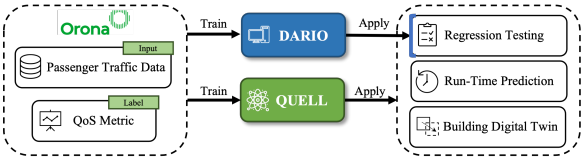


Figure 2: Industrial Context of QUELL

While classical regression test oracles are affordable in the context of the SiL test level, when considering full-day traffic profiles at the HiL test level, this testing does not scale due to the need to re-execute the previous version (note that at HiL, the test execution is real-time). Moreover, at operation time, it is not possible to re-execute any test. Because of this, recently, the ML-based oracle $\text{DARIO}_{\text{PRED}}$ [19] (i.e., the prediction component of the framework DARIO) has been proposed to determine, in an efficient manner, whether a system of elevators gives an adequate QoS.

As explained in Sect. 1, it is important to use these ML-based oracles in operation to perform additional activities like runtime monitoring, runtime performance prediction, etc. However, $\text{DARIO}_{\text{PRED}}$ is not applicable, as its ML models are trained on many features unavailable at operation time. Indeed, some features, like the passengers' weights, may not be obtainable at runtime. Moreover, the configuration of elevators in different buildings varies, and the number of available features for training ML models varies.

Thus, there is a need to learn an ML model with fewer features than what is possible at the design time while, at the same time, achieving model prediction performance comparable to that obtainable with many features. To this end, we explore the use of QELM to train an ML model with fewer features for which it is possible to extract the data from the real operation of elevators. We aim to demonstrate that even with few features, we can predict AWT with comparable prediction performance as with a complete feature set that is possible during design time. We also demonstrate that $\text{DARIO}_{\text{PRED}}$ cannot perform well with fewer features.

Fig. 2 shows the industrial context of QUELL, where the aim is to support regression testing of the dispatching algorithm. Regression testing at Orona is currently done with $\text{DARIO}_{\text{PRED}}$. Another possible application of QUELL is supporting real-time analyses

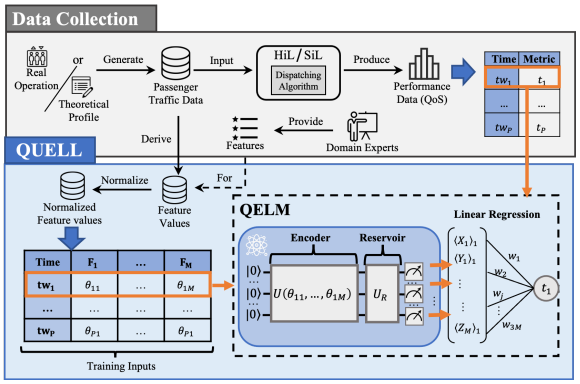


Figure 3: Overview of QUELL

by enhancing digital twins of industrial elevators previously built in [56], and other real-time predictions.

4 APPROACH

In this section, we first present the overview of QUELL. Next, we describe two hardware-efficient quantum encoders and the four quantum reservoir circuits we use in this paper.

4.1 Overview

QUELL leverages the capability of QELMs to train the model, which is a hybrid quantum-classical approach. Fig. 3 shows the overview of the data collection process and the training process of QUELL. To train the model, QUELL requires the previously collected data (i.e., *passenger traffic data* from, e.g., the *real operation* of industrial elevators) corresponding to domain-specific features developed by *domain experts* (elevator designers in our context), and the *QoS data* (AWT in our context) corresponding to QoS metrics that indicate the performance of an elevator. The elevator dispatching algorithm takes input passenger traffic data such as call floor, destination floor, traveling distance, and floor position. During design and development, such data can be simulated or obtained from theoretical traffic profiles and used as input data. During operation, such data is recorded from how real passengers used the elevators. However, during operation time, we can only record limited data, e.g., we can not record individual passengers' weight. In addition, data corresponding to QoS metrics can be extracted from simulations (e.g., in SiL or HiL setup). For SiL, this is achieved by feeding traffic data based on theoretical traffic profiles or real passenger traffic data to the Elevate simulator and obtaining such QoS values as the model's labels (i.e., *training targets*). In the HiL setup, hardware is involved in the simulation to obtain the QoS values.

Domain experts, i.e., elevator designers from Orona, provide the *domain-specific features* for training $\{F_1, \ldots, F_M\}$ as shown in Fig. 3. Next, *feature values* within each time window tw_j are obtained. There are two ways to obtain such values. First, they can be obtained via telemetry through the dispatching algorithm in operation. An alternative is to obtain such values from simulations or theoretical traffic profiles, which is common in Orona during design time. However, in the context of this paper, we focused only on the real operational data. Suppose we obtain the *feature values* of in total P number of time windows. For each time window, we consider the values as a M-dimension input vector $\boldsymbol{S}_j = [s_{j1}, \ldots, s_{jM}]$ for the model. The corresponding target output value is the functional QoS metric t_j obtained in the same time window tw_j. However, we cannot directly input the raw values into the quantum encoders of QELMs since these values are classical data and must be encoded to be processed by quantum circuits. Such transformation is performed by encoders (see details in Sect. 4.2). The encoders process information by parameterizing the angles of Pauli gates with input data. As a result, we need to consider the periodicity of the angle to avoid encoding several different values in the same quantum state. Arbitrary values will cause inaccurate encoding. Thus, we use min-max normalization on all datasets to normalize all the data, feature by feature, into the range of radians from 0 to π. The normalized feature data of the j-th time window is represented as $\boldsymbol{\theta}_j = [\theta_j^1, \ldots, \theta_j^M]$.

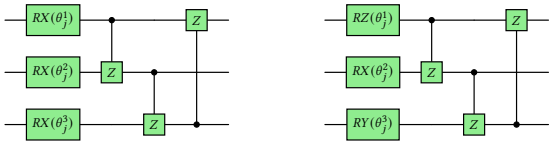
Next, we feed the normalized values $\boldsymbol{\theta}_j$ into a quantum circuit for each time window. Each qubit of a quantum circuit is first initialized with $|0\rangle$ state. Next, we inject $\boldsymbol{\theta}_j$ into the encoder circuit to transform the classical data into a quantum state. This generated quantum state, which is a hidden state, is the input of the quantum reservoir (see details in Sect. 4.3). The quantum state evolves inside the quantum reservoir circuit until it is measured. To extract partial information from the x, y, and z axis, we measure the expectation values with all three Pauli operators (i.e., X, Y, Z) on each qubit. Thus, we obtain the output vector of the j-th time window as $\boldsymbol{V}_j = [\langle X^1 \rangle_j, \langle Y^1 \rangle_j, \langle Z^1 \rangle_j, \ldots, \langle X^M \rangle_j, \langle Y^M \rangle_j, \langle Z^M \rangle_j]$.

After feeding feature values of all P time windows of the training datasets into the quantum circuit and getting all the measured output vectors, we train the linear regression model to optimize the weights $\boldsymbol{W} = [w_1, w_2, \ldots, w_{3M}]$ of the output function. We try to minimize the difference between the prediction t_j^{pre} and the corresponding training target t_j with a loss function Residual sum of squares (RSS). Thus, we minimize the loss with the equation.

$$RSS = \sum_{j=1}^{P} (\boldsymbol{W}\boldsymbol{V}\boldsymbol{j} - t_j)^2 \quad (1)$$

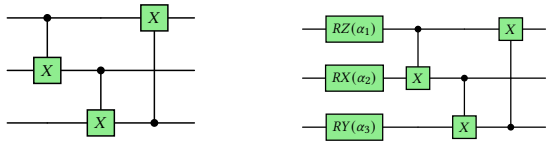
4.2 Quantum Encoders

We use two quantum encoders to transfer classical data into quantum states before using it in a quantum reservoir circuit to evolve the state. Since hardware restrictions exist on current quantum computers (limited connections among qubits, a limited number of native gates, and the decoherence issue), quantum circuits with high-depth and dense connections are unsuitable. To this end, a class of hardware-efficient ansatz (a parameterized quantum circuit to approximate with a defined sequence of gates to approximate a problem) is proposed [28]. This class of circuits has been used to encode classical data for quantum machine learning [9, 42]. It typically contains a series of gates, including single-qubit rotation gates followed by some two-qubit entanglement gates. The circuit can be composed of single or multiple layers (i.e., depth) by repeating this set of gates. Also, single or multiple qubits can encode one feature value depending on the size of the problem and the number of available qubits. A higher number of qubits encoding features leads to a higher dimension of the Hilbert space, which adds more dynamics to the circuit. In contrast, a lower number may lack such extensive dynamics, leading to higher efficiency and cost-saving.



(a) DHE **encoder** **(b)** RHE **encoder**

Figure 4: Two types of quantum encoders used in QUELL



(a) CNOT **reservoir** **(b)** Rotation **reservoir**

Figure 5: Two of the four quantum reservoirs used in QUELL

The following two encoders exhibit a common gate structure but vary in implementing single qubit rotation gates.

Determined hardware efficient encoder (DHE**):** In each layer, each qubit is first applied with an *RX*, and the rotation angles are determined by input values. In our case, for single qubit encoding, each *RX* gate is parameterized by one normalized feature value. Specifically, to encode data in the j-th time window, i.e., $\boldsymbol{\theta}_j$, the rotation of *RZ* gate on the k-th qubit should be determined by θ_j^k, which is the normalized value of the j-th feature. In the following, *CZ* gates are applied with a cyclic entangling structure, where qubits are connected cyclically with successive *CZ* gates. An example of one-layer quantum circuit is shown in Fig. 4a, where, given three features selected by domain experts $\{F_1, F_2, F_3\}$, we encode the values of the j-th time window $\boldsymbol{\theta}_j = [\theta_j^1, \theta_j^2, \theta_j^3]$ in order on *RX* gates for each qubit, followed by the cyclic entangling *CZ* gates.

Randomized hardware efficient encoder (RHE**):** This encoder follows the same main structure of the DHE encoder. The difference is that the *RX* gates of DHE are replaced by randomly selected single qubit rotation gates among *RX*, *RY*, and *RZ* gates. For example, the encoding of the classical data follows the same rule. With the same example illustrated in DHE, given the same three features in the j-th time window and corresponding values $\boldsymbol{\theta}_j = [\theta_j^1, \theta_j^2, \theta_j^3]$, Fig. 4b shows one possible RHE circuit of one layer.

4.3 Quantum Reservoir

After obtaining the state vectors from the encoders, we use the quantum reservoir circuits to evolve the quantum state by executing the quantum circuit until it is measured. In the following, we introduce four quantum reservoir circuits used in the paper.

CNOT reservoir (CNOT**):** It comprises *CX* gates (i.e., *CNOT*) without randomly assigned parameters. *CX* gates are applied similarly to the cyclic entangling structure implemented in the encoder circuits. Fig. 5a shows an example of CNOT reservoir with 3 qubits.

Harr random reservoir (Harr**):** It uses a single random operator as the quantum reservoir circuit. The unitary matrix of the operator is sampled from Haar measure [34].

Ising Mag Traverse Reservoir (ISING**):** It is one of the widely studied reservoirs. It has a fully connected transverse field, Ising Hamiltonian, in the form of Ising model (a binary quadratic mathematical model). The unitary operator is defined as:

$$U_{\texttt{ISING}} = \exp(-iH\Delta t)$$

where i is the imaginary number, H is the Hamiltonian, and Δt is the time step of the Hamiltonian. The Hamiltonian is defined as

$$H = \sum_{k,j} J_{k,j} Z_k Z_j + \sum_j a_j X_j \quad (2)$$

where coefficients a_j and $J_{k,j}$ are the randomly sampled parameters in the unitary matrix, and X_j and Z_j represent the application of the Pauli-X and Pauli-Z gate on the j-th qubit.

Rotation reservoir (Rotation**):** It has single rotation gates and cyclic structure *CX* gates. Each qubit has one single-qubit rotation gate randomly selected within *RX*, *RY*, and *RZ* gates. The rotation angles are also randomly assigned. Next, cyclic-structured *CX* gates are applied. Fig. 5b shows a possible circuit with three qubits.

5 EXPERIMENT DESIGN

Research questions. We investigate the effectiveness of QUELL using the following research questions (RQs). The detailed experiment results can be found online at [1].

RQ1. Which combination of encoder and reservoir of QUELL achieves the best prediction performance with a different number of features?
Given that there are two possible encoders and four possible reservoirs, we aim to find their best combination for QUELL, that can be used for the experiments in the other RQs.

RQ2. By using the optimal combination of encoder and reservoir, what is the minimum number of features for which QUELL achieves a prediction performance comparable to that achievable using the maximum number of features?
In RQ1, we find the optimal combination of encoder and reservoir for QUELL. In this RQ, by using this combination, we aim to find the minimum number of features required by QUELL to achieve a prediction performance at least similar to that obtainable by using more features. This RQ helps evaluate the prediction performance of QUELL when using a limited number of features.

RQ3. How well does QUELL perform compared to the baseline when using different numbers of features for predictions?
This RQ studies whether QUELL can help achieve, at least, similar performance with fewer features compared to a classical regression machine learning algorithm.

Features and datasets. We use the average waiting time (AWT) as the performance metric, representing the QoS of elevators. For evaluation, Orona provided 12 relevant features for training the model in QUELL. These features are: F_1. Number of upward calls from low-level floors. F_2. Number of upward calls from medium-level floors. F_3. Number of upward calls from high-level floors. F_4. Number of downward calls from low-level floors. F_5. Number of downward calls from medium-level floors. F_6. Number of downward calls from high-level floors. F_7. Average distance of the travel from the upward calls. F_8. Average distance of the travel from the downward calls. F_9. Number of total upward calls in the past 5 minutes. F_{10}. Number

Table 2: Feature sets *FS*

Feature set (*FS*)		Selected Features
Number of features	ID	
2	FS_2	F_{11}, F_{12}
3	FS_{3a}	F_{11}, F_{12}, F_7
	FS_{3b}	F_{11}, F_{12}, F_1
4	FS_4	F_{11}, F_{12}, F_7, F_8
5	FS_5	$F_{11}, F_{12}, F_7, F_8, F_1$
10	FS_{10}	$F_1, F_2, F_3, F_4, F_5, F_6, F_7, F_8, F_9, F_{10}$

of total downward calls in the past 5 minutes. F_{11}. Number of calls going upwards F_{12}. Number of calls going downwards.

To assess the feasibility of training models with a varied number of features, Orona provided us with specific *feature sets* (*FS*) built using the 12 features. Table 2 reports the details of the feature sets. In the table, FS_n represents a feature set with n features, with $n \in \{2, 3, 4, 5, 10\}$; we use *3a* and *3b* to distinguish the two sets of 3 features. Feature sets were built according to these observations. As elevators of the same product line have different configurations, some of them may lack the ability to record detailed information on elevator calls, such as starting and destination floors. Thus, features F_1 to F_6 are not in feature sets FS_2, FS_{3a}, FS_4, and FS_5. In FS_{3b}, instead, only F_1 is present. In addition, since F_7 and F_8 are not available at runtime during operation, these features are not present in FS_2 and FS_{3b}. FS_{10} is the feature set used to train models of the baseline DARIO$_{\text{PRED}}$ in [19].

Orona provided us with 4 datasets of information extracted from the real operation of elevators installed in a 10-floor building: it is the passenger traffic data and the corresponding `AWT` values for 4 days. According to the passenger traffic data, we obtained information on the features for a time window of 5 minutes. We used an elevator simulator, i.e., the one provided by ELEVATE [15], and used in Orona. The simulator was executed over each dataset, and the `AWT` in each time window was obtained as the target for training.

Independent variables. The first independent variable is `Encoder` type, which takes values in {`DHE`, `RHE`}. The second independent variable is `Reservoir` type, which takes values in {`CNOT`, `Harr`, `ISING`, `Rotation`}. We investigate which of the 8 combinations of `Encoder` and `Reservoir` (`Encoder_Reservoir`) gives the best performance for QUELL for different feature sets with different number of features (see Table 2), and different datasets. We also compare QUELL with the baseline approach DARIO$_{\text{PRED}}$, equipped with the two best ML models in [19] (i.e., SVM and regression tree).

In our study, we use cross-validation, where each dataset was treated as the test dataset while the remaining 3 datasets served as the training dataset. Thus, we conducted four experiments for every feature set FS_n and combinations of `Encoder_Reservoir`. The experiments were labeled using $ExpDay_n$, with $n \in \{1, 2, 3, 4\}$, to denote the four datasets provided by Orona.

Parameter settings. For feature normalization, we used the following range $[0, \pi]$. For other parameters, we used default parameter settings, i.e., encoder depth set to 1, one qubit for each feature, and depth of `CNOT` and `Rotation` reservoirs set to 10. We repeat each experiment 30 times to account for randomness [4]. As parameter values for two versions of the baseline DARIO$_{\text{PRED}}$, we use the same used in the original study [19].

We implemented QUELL with Qreservoir package [40] of framework Qulacs [50], together with its quantum simulator. Experiments were run on an AMD EPYC 7601 32-core processor.

Evaluation metrics and statistical tests. For RQ1, for each dataset $ExpDay_i$, feature set FS_j, and combination `Encoder_Reservoir`, we assessed the prediction quality using the *mean squared error*:

$$MSE = \frac{1}{P}\sum_{j=1}^{P}(t_j^{pre} - t_j)^2$$

where t_j^{pre} represents the target value predicted by the model. This indicates the prediction quality of combination `Encoder_Reservoir` for dataset $ExpDay_i$, using feature set FS_j. Since we repeated experiments 30 times, we obtained 30 *MSE* values $MSE_1, \ldots, MSE_{30}$. We computed *average MSE* value as $AMSE = \sum_{i=1}^{30} MSE_i/30$.

Then, for each FS_j in $ExpDay_i$, we ranked the eight combinations `Encoder_Reservoir` based on *AMSE* values. Finally, for all FS_j in all datasets, we picked the pair with the highest rank in most cases.

For RQ2, we compared the performance of QUELL across all FS_j in terms of *MSE*. First, we performed the Kruskal–Wallis test to determine whether overall differences exist for the 30 runs across all the FS_j. If the test reveals a p-value less than 0.05, it means that there are significant differences among some pairs of FS_j. In that case, we compared all pairs of FS_j with the Mann–Whitney U test combined with $\hat{A}_{12}$ as the effect size. If the p-value of a pair, e.g., FS_2 and FS_{10} is less than 0.05, then it means that significant differences exist between them. Since we are performing multiple comparisons, we also corrected p-values with the Holm–Bonferroni method. Finally, the strength of significance was tested with the $\hat{A}_{12}$ statistics. An $\hat{A}_{12}$ value lower than 0.5 means that the likelihood that the first pair is better than the second one, e.g., FS_2 is better than FS_{10}, and vice versa. According to [30], the effect size can be interpreted as: *Small* in $(0.34, 0.44]$ and $[0.56, 0.64)$; *Medium* in $(0.29, 0.34]$ and $[0.64, 0.71)$; *Large* in $[0, 0.29]$ and $[0.71, 1]$.

For RQ3, we compared QUELL trained over each feature set FS_i ($i \in \{2, 3a, 3b, 3, 5, 10\}$), with the two best classical ML algorithms in DARIO$_{\text{PRED}}$ [19] trained with FS_{10}. In this case, we compared 30 *MSE* values of QUELL with one value of DARIO$_{\text{PRED}}$. Thus, we performed a one-sample Wilcoxon test to compare a sample with one value. As an effect size measure, we used Cohen's d [11] and used an existing guide to interpret its magnitude as follows: *Small* if $0 < d < 0.2$ or $-0.2 < d < 0$; *Medium* if $0.2 \leq d \leq 0.8$ or $-0.8 \leq d \leq -0.2$: *Large* if $d > 0.8$ or $d < -0.8$. If d is lower than 0, it means that QUELL is better than DARIO$_{\text{PRED}}$ (i.e., lower *MSE* values). Otherwise, d greater than 0 means that QUELL is worse.

6 RESULTS AND ANALYSES

RQ1: Assessment of Encoding and Reservoirs

First, we evaluate the performance of the 8 combinations of `Encoder_Reservoir`. To analyze the overall performance of each combination through various feature sets *FS* for all experiments *ExpDay*, we calculate the *AMSE* of 30 runs for each *setting*, i.e., pair of feature set *FS* and dataset *ExpDay*. Since there are 6 feature sets and 4 datasets, there are 24 settings. Fig. 6 reports violin plots where each data

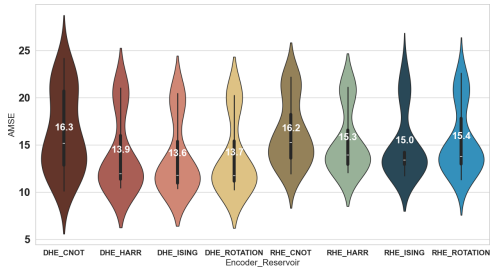


Figure 6: RQ1 – *AMSE* of 8 combinations `Encoder_Reservoir`

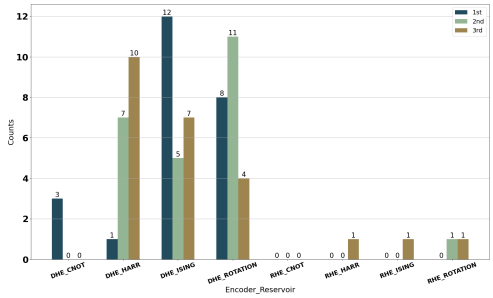


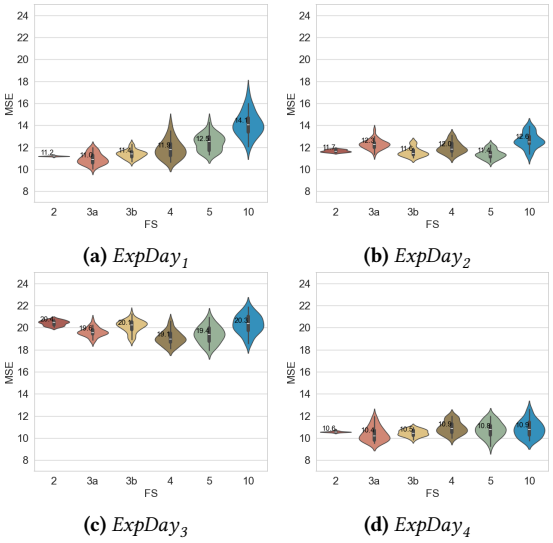
Figure 7: RQ1 – 1st, 2nd, and 3rd position for each combination of `Encoder_Reservoir`

point represents the *AMSE* for each setting, and each violin represents the performance of each combination `Encoder_Reservoir`. We observe that most *AMSE* values range from 10 to 25, and that the values of `DHE _Harr`, `DHE _ISING`, and `DHE _Rotation` are the best (i.e., lower than the other ones).

There are also two outlier data points for `RHE _ISING` and one for `RHE _Rotation` (not shown in the figure). We closely examine the *MSE* values generated by each run with various settings. We identify the runs that produced *MSE* values exceeding 25, resulting in 14 runs. Among them, 11 were generated by the `RHE _ISING` combination, while the remaining three were produced by `RHE _Rotation`. This observation suggests that these two combinations are unstable. As detailed in Sect. 4, the `RHE` encoder is randomly generated. Furthermore, the randomly chosen Hamiltonian in the `ISING` model and the randomly selected rotation gates (along with their angles) in `Rotation` may contribute to such instability. Hence, it is highly likely that these random aspects contributed to a sub-optimal reservoir circuit, resulting in unstable prediction performance.

To analyze the performance of the combinations in detail, we rank the *AMSE* of each combination to obtain the top three combinations for each setting. We report the results in Fig. 7. We notice that in most settings, `DHE` encoder is in the 1st, 2nd, and 3rd combination. `DHE _Harr`, `DHE _ISING`, and `DHE _Rotation` ranked top 3 in the majority of settings. Finally, we observe that `DHE _ISING` ranks the first in 12 out of 24 settings.

Conclusions for RQ1: Overall, the `ISING` reservoir combined with the `DHE` encoder enables QUELL to perform the best. Hence, we suggest using this combination in QUELL.



(a) $ExpDay_1$ **(b)** $ExpDay_2$

(c) $ExpDay_3$ **(d)** $ExpDay_4$

Figure 8: RQ2 – *MSE* of QUELL with different *FS*

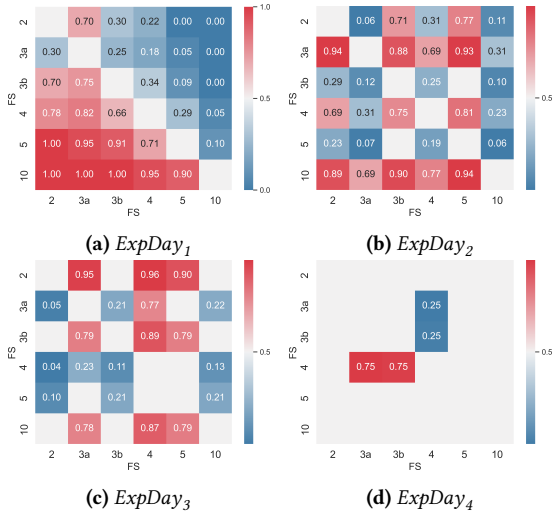
RQ2: QUELL's Performance with Different Feature Sets

We choose the best combination of `Encoder_Reservoir` in RQ1, i.e., `DHE _ISING`, to compare the performance of QUELL with different feature sets *FS* of different sizes. For each dataset, we draw the violin plot of *MSE* values shown in Fig. 8. We notice that the fluctuation in *MSE* values produced by QUELL decreases with smaller feature sets, as shown by the low variance of FS_2 in all four datasets. The possible reason is that, since the `DHE` encoder remains constant for a specific feature set, the randomness is only introduced by the quantum reservoir, specifically by the unitary matrix determined by the `ISING` Hamiltonian. Because fewer features require few qubits, according to Eq. 2, there is a reduced need for parameters to be randomly selected.

We also observe that QUELL with less than 10 features can consistently achieve performance comparable to QUELL with 10 features (i.e., similar or even lower *MSE* values). Since 10 features require 10 qubits, this increases the likelihood that random parameters are assigned to the `ISING` Hamiltonian; this can generate a suboptimal reservoir circuit, which may exhibit limited dynamics, potentially leading to a less effective mapping from the classical data into the observed values. Moreover, since we aim to use the most efficient quantum circuit, the encoder depth qubit is 1. However, in the case of complex quantum circuits involving 10 qubits, a higher depth may demonstrate increased effectiveness.

For $ExpDay_3$, the *MSE* values are generally higher than in other experiments. The cause of this is that the testing dataset includes an outlier with an `AWT` value exceeding 35 seconds, whereas the `AWT` values in the training dataset fall within the range of 0 to 26 seconds, which results in difficulty for QUELL to predict that value.

We have also used statistical tests to analyze the comparison among those configurations in detail. First, for each dataset, we applied the Kruskal-Wallis test among the different versions of



(a) $ExpDay_1$ **(b)** $ExpDay_2$

(c) $ExpDay_3$ **(d)** $ExpDay_4$

Figure 9: RQ2 – Comparison of QUELL with different *FS* ($\hat{A}_{12}$ values of the comparison of the configuration on the y-axis vs. the configuration of the x-axis)

QUELL trained with different feature sets *FS*. We found that all p-values are lower than 0.05, meaning there are significant differences among *MSE* values of different *FS*. Then, we compared the *MSE* values of each pair of configurations using the Mann–Whitney U test, together with Holm-Bonferroni correction according to Sect. 5. In Fig. 9, gray cells indicate no significant difference between the corresponding two configurations. We also report the $\hat{A}_{12}$ values for the other cells. Blue color (corresponding to $\hat{A}_{12}$ lower than 0.5) means that the configuration on the y-axis is better (i.e., lower *MSE*) than the configuration on the x-axis; red color means the opposite. In $ExpDay_1$, QUELL with FS_{3a} is the most competitive. In $ExpDay_2$ and $ExpDay_3$, no configuration achieves significantly better performance than QUELL with FS_5. In $ExpDay_3$, QUELL with FS_4 is the most competitive configuration. However, in $ExpDay_4$, there is no significant difference among most configurations. Moreover, QUELL with FS_{10} is significantly worse than any other *FS* in $ExpDay_1$ and $ExpDay_2$. In all four experiments, QUELL with FS_{10} is not significantly better than any other configuration, which aligns with the previous finding that it requires further setting.

Conclusions for RQ2: Overall, QUELL with few features outperforms QUELL with the maximum number of features 10. This shows the effectiveness of QELM in our industrial context, where few features are available for real-time predictions.

RQ3: Comparison of QUELL with DARIO$_{\text{PRED}}$

We compare the *MSE* values of all the versions of QUELL (i.e., using all the feature sets *FS*) with those of the baseline DARIO$_{\text{PRED}}$ using the feature set with 10 features, i.e., FS_{10}. For DARIO$_{\text{PRED}}$, we consider its two best versions in [19], i.e., when using SVM and regression tree. We identify the two versions as DARIO$^{\text{SVM}}_{\text{PRED}}$ and DARIO$^{\text{RT}}_{\text{PRED}}$. Since the *MSE* values of DARIO$^{\text{SVM}}_{\text{PRED}}$ and DARIO$^{\text{RT}}_{\text{PRED}}$ are fixed for the same training and testing datasets, we perform a one-sample Wilcoxon signed rank test to compare *MSE* values of QUELL with various *FS* with that of DARIO$^{\text{SVM}}_{\text{PRED}}$ and DARIO$^{\text{RT}}_{\text{PRED}}$ in each experiment *ExpDay*. The results show that all p-values are less than 0.05, which indicates the significant difference between QUELL and the two classical algorithms of DARIO$^{\text{SVM}}_{\text{PRED}}$ and DARIO$^{\text{RT}}_{\text{PRED}}$.

Next, we compute Cohen's *d* effect size to see the magnitude of the differences. The results show that all calculated *d* values are lower than −1, which indicates that all *MSE* values generated by our approach are greatly smaller than those generated by DARIO$^{\text{SVM}}_{\text{PRED}}$ and DARIO$^{\text{RT}}_{\text{PRED}}$ in each experiment. As a result, we conclude that QUELL with any number of features performs significantly better than the state of the practice classical machine learning algorithms.

For each experiment, we also counted the number of runs that the *MSE* values generated by QUELL with different *FS* are larger than that of DARIO$^{\text{SVM}}_{\text{PRED}}$ and DARIO$^{\text{RT}}_{\text{PRED}}$. For $ExpDay_1$ and $ExpDay_2$, all *MSE* values generated by QUELL with all feature sets, are smaller than those of both DARIO$^{\text{SVM}}_{\text{PRED}}$ and DARIO$^{\text{RT}}_{\text{PRED}}$. For $ExpDay_3$ with FS_{10}, there are two runs in which the generated *MSE* values of QUELL are larger than the ones generated by DARIO$^{\text{SVM}}_{\text{PRED}}$. As in RQ2, QUELL with FS_{10} is the worst configuration for this dataset. In $ExpDay_4$, for QUELL with FS_{3a}, FS_4, FS_5, FS_{10}, there are 1, 6, 3, and 4 out of 30 runs in which QUELL generated *MSE* values higher than the ones generated by DARIO$^{\text{RT}}_{\text{PRED}}$.

These results align with RQ2's results. For example, in $ExpDay_3$, we can see that QUELL with FS_{10} generated the highest *MSE* value according to the violin plot in Fig. 8c, which provides the explanation of the two runs in which it is worst than DARIO$^{\text{SVM}}_{\text{PRED}}$. Moreover, as shown in Fig. 9d, in $ExpDay_4$, in general, QUELL with all feature sets has similar performance, except for QUELL with FS_4, which is significantly worse than FS_{3a} and FS_{3b}. This observation explains why QUELL with FS_4 contains the highest number of runs (6) in which *MSE* values of QUELL are higher than those of DARIO$^{\text{RT}}_{\text{PRED}}$.

In any case, in most of the cases, QUELL performs better than the baselines (i.e., it has lower *MSE* values).

Conclusions for RQ3: For the same prediction task in our industrial context, QUELL outperforms classical machine learning approaches (i.e., DARIO$^{\text{SVM}}_{\text{PRED}}$ and DARIO$^{\text{SVM}}_{\text{PRED}}$), by using less features. This demonstrates the potential of QELM.

7 DISCUSSION

We present a discussion and key lessons learned in this section that are valuable for practitioners and researchers.

7.1 Improvement Over State of the Practice

The comparison with the state of the practice approach DARIO$_{\text{PRED}}$ demonstrated that QELM, indeed, helps our approach perform significantly better than classical machine learning algorithms, thus proving evidence of improvement. We further compared time cost associated with QUELL and classical machine learning algorithms. The training time, even on the simulator, was around 1 second or even lower, whereas the inference time was much lower than 1 second for the four datasets we experimented with. Such time cost is practically negligible in our context and, so, shows that even with a negligible cost QUELL can perform better than the state of the practice approach. Notice that executing QELM on a real quantum

computer will be much faster than the simulators, thereby bringing down these costs even further.

7.2 Potential Applications in Orona

QUELL demonstrated a good performance at predicting AWT over 5-minute time windows with few features. As discussed before, Orona's main application will be runtime monitoring. Orona continuously maintains its code and deploys changes in operation. This code may have bugs that lead to sub-optimal elevator scheduling, incrementing passenger waiting times and so reducing comfort. QUELL will help Orona effectively and efficiently predict waiting times at runtime, serving as a runtime monitor to take corrective actions (e.g., roll-back to a previous version) when an issue is found.

Besides this, QUELL can have other applications in Orona. For instance, it can be used for regression testing at the HiL test level when not all features can be obtained. The classical machine learning algorithms employed in the state-of-the-practice approach [19] could be replaced with QELM for this purpose.

Another possible application is the deployment of QUELL in digital twins (DTs) of elevators. In our recent work with Orona [56], we built DTs of elevators with classical machine learning algorithms. Also, we supported the evolution of DTs in response to the evolution of elevators. These DTs use neural network models for waiting time predictions, which is a task similar to that of this work. This prediction task could also benefit from QELM.

7.3 Generalizability to Other Practical Contexts

We demonstrated the application of QELM in one industrial context for one prediction task. However, it is important to note that our implementation of QELM is not specific to this industrial context, and it can also be applied to other prediction tasks in other industrial contexts with different feature sets. Whether the current best combination of encoding type and reservoir types will work for other contexts requires further investigation. However, we find that the current demonstration of QELM in our industrial context is representative, demonstrating the benefits of QELM. Our work could inspire practitioners and researchers to experiment with QELM and other quantum machine learning techniques to overcome software engineering problems that require regression problems and the number of features to be used is limited.

7.4 Research Implications

We discuss research implications for classical and quantum software engineering, and for theoretical foundations of QELM.

Classical and Quantum Software Engineering. Our work opens up two new research directions within the software engineering area (both classical and quantum). **First**, it opens up areas in applied quantum machine learning (QML) in software engineering applications. In particular, QELM and other new QML algorithms can be studied for various classical software engineering problems, such as regression testing, defect prediction, software development cost prediction, and software performance predictions. Similarly, even though quantum software engineering (QSE) methods [3, 58] are limited, QML methods can be applied to solve QSE problems such as quantum software testing and debugging. Since quantum software operates on quantum states, there might not be a need for quantum encoders. **Second**, we also see the opportunities for empirical research focusing on empirically evaluating various parameters of QELM and other QLM algorithms. For instance, for QELM, we used default settings for parameters such as encoder depth equal to 1 and one feature mapped to exactly one qubit. There is a need for more extensive empirical evaluations to study how varying these parameters affect the prediction performance; hence, more carefully planned empirical studies are required. Such empirical evaluations, for example, will provide evidence about the performance of various parameter configurations for different problems. Such evidence will also result in developing guides (currently missing) for conducting empirical evaluations for QELMs, guiding users, for example, about which settings to use, which statistical tests to perform, and which cost-effectiveness criteria to use.

Theoretical Foundations in QELM. This paper applied existing QELM implementations (including encoding and reservoir types) to our industrial problem. Such QELM implementation is preliminary, and more research is needed to design better encoders and reservoirs. One potential direction to investigate is to design domain-specific encoders and reservoirs that work for specific domains of problems (e.g., elevator operation optimization). For further discussion on the research opportunities offered by QELM, we refer interested readers to [36].

7.5 Limitations

Our application has some limitations, given that QELM is a very new area. First, we executed QELM on a quantum computer simulator with Qulacs framework. We used an ideal simulator, i.e., with no hardware noise. However, currently, quantum computers have hardware noise that affects the computations they perform. As a result, if we run QELM on a real quantum computer, our results will be affected. However, currently, it is not possible to directly execute QELM on real quantum computers due to practical hardware constraints. Moreover, we would like to highlight that QELM is one of the algorithms inherently designed to deal with hardware noise; thus, we expect them to perform well even on real quantum computers. On the other hand, note that as the number of qubits increases, the simulation time on quantum computer simulators increases exponentially, thereby limiting the practical application of QELM. Nonetheless, once QELM can be executed on quantum computers, the execution time, even with a higher number of qubits, will be much lower than execution on simulators. In addition, to deal with slow QELM execution on the simulator, we implemented quantum encoders and reservoirs with low complexity (e.g., each feature is encoded as one qubit) for efficient execution. However, in the future, with the increased accessibility of real quantum computers with a higher number of qubits, we can build more complex quantum reservoirs for QUELL to enhance prediction accuracy.

8 THREATS TO VALIDITY

Internal validity: We normalize the features in the range of $[0, \pi]$. Different normalization ranges may affect the results, thereby requiring additional investigation. QELM's parameters were set to default values, i.e., encoder depth to 1, one qubit mapped to one feature, and depths of quantum reservoirs to 10. Such default values have shown good results in the previous studies [21]. Finally,

to select the best combination of encoding and reservoir type for QUELL, we performed a systematic experiment as reported in RQ1.

For $\text{DARIO}_{\text{PRED}}$ used as the baseline, we picked its settings from the original work [19], i.e., the maximal number of decision splits per tree of 25 for the regression tree, whereas the Gaussian kernel function was used. Other parameters were set to default settings. For comparing QUELL with the baselines, we picked the two best machine learning algorithms from the original work [19].

Conclusion validity: There is inherent randomness in QELM; thus, we repeat each testing task 30 times to account for it, as common practice in evaluating randomized algorithms [4]. We collected the results of the 30 repetitions and applied appropriate statistical tests based on established guides from the literature [4].

External validity: We experimented with only four datasets corresponding to four days of elevator operations in one building. Naturally, it threatens the generalizability of our results since data for different building configurations, days, and different configurations of the dispatching algorithm are possible. This dataset, however, was obtained from a real building and is a good representation of the real world. Access to this data is difficult even for Orona's engineers as it requires significant resources. Once we have access to more data, we can easily extend the empirical evaluation.

9 RELATED WORKS

Multiple recent studies focused on enhancing the software quality of elevator dispatching algorithms, covering robustness analysis [23, 24], testing [6–8, 18, 19], debugging [52] and repair [53]. Our work is close to those from Ayerdi et al. [7, 8], since, given a passenger set, predicting which the output should be is not trivial. Two key differences exist between this work and those from Ayerdi et al. [7, 8]. Firstly, our approach aims to predict the waiting time of the system by applying QELM, whereas their approach leverages metamorphic testing. Secondly, our approach is applicable both at design-time and operation-time, whereas their technique is solely focused on operation-time. Similar to our approach, others aim at predicting waiting times of passengers [5, 19]. Nevertheless, as explained in previous sections, their approach has certain limitations that we overcome in this paper by using QELM instead of classical ML-based techniques. Specifically, QELM can provide advantages when not all features are available, which is the case for runtime monitoring of these systems when deployed in operation. In summary, to the best of our knowledge, this is the first paper that applies QELM to predict QoS metrics of a system of elevators.

In the past, several studies tackled the test oracle problem by employing machine learning by following different strategies (e.g., automatically predict the test verdict [10, 20, 31, 43]). Similar to our approach, several studies aim at predicting which is the expected output [2, 12, 27, 32, 35, 41, 44–47, 54, 57]. There are significant differences between our approach and these ones. The first one is that we are applying QELM instead of traditional machine-learning techniques (e.g., SVMs) to reduce the feature space so that our technique can be applicable at the operation time. The second one is that we apply these techniques in the context of Cyber-Physical Systems (CPSs), unlike most of the aforementioned approaches, which focus on testing software code. Lastly, unlike the previously mentioned approaches, we apply our approach in the context of a real industrial case study system. In the context of CPSs, and more particularly autonomous driving systems, several approaches have targeted runtime monitoring by leveraging advanced techniques to measure the uncertainty of the DNNs taking control of the vehicle [48, 49]. Unlike our industrial system, such approaches are applicable for cases where the system's control is driven by ML-based techniques, which is not the case in our industrial system.

Quantum reservoir computing (QRC) and QELM offer ease in model training with quantum reservoir circuits for linear regression models for machine learning tasks (e.g., prediction) [17, 36, 51]. The difference between QRC and QELM is that tasks accomplished by QRC employs the memory of the reservoir, while QELM uses memoryless reservoirs, which makes the training of QELM easier [36]. Given that these techniques can potentially be executed on quantum computers with high computational power complemented with the increased availability of quantum computing platforms, there has been increased interest in using QRC or QELM for solving problems in the classical domain [16, 22, 33, 37, 38, 55] and quantum domain [13, 14, 21, 26, 29]. However, these works focus on benchmark problems. In contrast, QUELL provides implementation of QELM for solving a practical industrial problem with real operational data from industrial elevators.

10 CONCLUSION

We presented an industrial application of quantum extreme learning machine (QELM) for solving the waiting time prediction task in the context of elevators. Using four real datasets from the elevators' real operation, we demonstrated that QELM could offer benefits by performing significantly better prediction even with fewer features than the classical ML algorithms that rely on a higher number of features. QUELL can also be applied to other similar prediction problems in other industrial contexts. In the future, we would like to experiment with our approach with more datasets and other application contexts (i.e., integration with digital twins of elevators and real-time prediction) in the context of Orona. Moreover, we will apply our approach to other industrial and real-world applications. Given that we ran our approach on quantum computer simulators with no noise, we will include noise on the encoder and reservoir to study how it affects our results. On the foundational side, we would also like to design new domain-specific encoders and quantum reservoirs for different applications.

ACKNOWLEDGMENTS

This work is supported by the Qu-Test project (Project#299827) funded by the Research Council of Norway. X. Wang is supported by Simula's internal strategic project on quantum software engineering. S. Ali acknowledges support from the *Quantum Hub initiative* (OsloMet). P. Arcaini is supported by Engineerable AI Techniques for Practical Applications of High-Quality Machine Learning-based Systems Project (Grant Number JPMJMI20B8), JST-Mirai. Aitor Arrieta is part of the Software and Systems Engineering research group of Mondragon Unibertsitatea (IT1519-22), supported by the Department of Education, Universities and Research of the Basque Country.

REFERENCES

[1] 2024. *Application of Quantum Extreme Learning Machines for QoS Prediction of Elevators' Software in an Industrial Context*. Zenodo. https://doi.org/10.5281/zenodo.11183939

[2] K. K. Aggarwal, Yogesh Singh, A. Kaur, and O. P. Sangwan. 2004. A neural net based approach to Test Oracle. *SIGSOFT Softw. Eng. Notes* 29, 3 (may 2004), 1–6. https://doi.org/10.1145/986710.986725

[3] Shaukat Ali, Tao Yue, and Rui Abreu. 2022. When Software Engineering Meets Quantum Computing. *Commun. ACM* 65, 4 (mar 2022), 84–88. https://doi.org/10.1145/3512340

[4] Andrea Arcuri and Lionel Briand. 2011. A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In *Proceedings of the 33rd International Conference on Software Engineering* (Waikiki, Honolulu, HI, USA) *(ICSE '11)*. Association for Computing Machinery, New York, NY, USA, 1–10. https://doi.org/10.1145/1985793.1985795

[5] Aitor Arrieta, Jon Ayerdi, Miren Illarramendi, Aitor Agirre, Goiuria Sagardui, and Maite Arratibel. 2021. Using machine learning to build test oracles: an industrial case study on elevators dispatching algorithms. In *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 30–39. https://doi.org/10.1109/AST52587.2021.00012

[6] Aitor Arrieta, Maialen Otaegi, Liping Han, Goiuria Sagardui, Shaukat Ali, and Maite Arratibel. 2022. Automating Test Oracle Generation in DevOps for Industrial Elevators. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 284–288. https://doi.org/10.1109/SANER53432.2022.00044

[7] Jon Ayerdi, Valerio Terragni, Aitor Arrieta, Paolo Tonella, Goiuria Sagardui, and Maite Arratibel. 2021. Generating metamorphic relations for cyber-physical systems with genetic programming: an industrial case study. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1264–1274. https://doi.org/10.1145/3468264.3473920

[8] Jon Ayerdi, Pablo Valle, Sergio Segura, Aitor Arrieta, Goiuria Sagardui, and Maite Arratibel. 2023. Performance-Driven Metamorphic Testing of Cyber-Physical Systems. *IEEE Transactions on Reliability* 72, 2 (2023), 827–845. https://doi.org/10.1109/TR.2022.3193070

[9] Kishor Bharti, Alba Cervera-Lierta, Thi Ha Kyaw, Tobias Haug, Sumner Alperin-Lea, Abhinav Anand, Matthias Degroote, Hermanni Heimonen, Jakob S Kottmann, Tim Menke, et al. 2022. Noisy intermediate-scale quantum algorithms. *Reviews of Modern Physics* 94, 1 (2022), 015004.

[10] Ronyérison Braga, Pedro Santos Neto, Ricardo Rabêlo, José Santiago, and Matheus Souza. 2018. A machine learning approach to generate test oracles. In *Proceedings of the XXXII Brazilian Symposium on Software Engineering* (Sao Carlos, Brazil) *(SBES '18)*. Association for Computing Machinery, New York, NY, USA, 142–151. https://doi.org/10.1145/3266237.3266273

[11] Jacob Cohen. 1969. *Statistical power analysis for the behavioral sciences*. New York: Academic Press.

[12] Junhua Ding and Dongmei Zhang. 2016. A Machine Learning Approach for Developing Test Oracles for Testing Scientific Software. In *The 28th International Conference on Software Engineering and Knowledge Engineering, SEKE 2016, Redwood City, San Francisco Bay, USA, July 1-3, 2016*, Jerry Gou (Ed.). KSI Research Inc. and Knowledge Systems Institute Graduate School, 390–395. https://doi.org/10.18293/SEKE2016-137

[13] L Domingo, G Carlo, and F Borondo. 2022. Optimal quantum reservoir computing for the noisy intermediate-scale quantum era. *Physical Review E* 106, 4 (2022), L043301.

[14] Laia Domingo, G Carlo, and F Borondo. 2023. Taking advantage of noise in quantum reservoir computing. *Scientific Reports* 13, 1 (2023), 8790.

[15] Elevate 2024. Elevate. https://peters-research.com/index.php/elevate/.

[16] Keisuke Fujii and Kohei Nakajima. 2017. Harnessing disordered-ensemble quantum dynamics for machine learning. *Physical Review Applied* 8, 2 (2017), 024030.

[17] Keisuke Fujii and Kohei Nakajima. 2021. Quantum reservoir computing: A reservoir approach toward quantum machine learning on near-term quantum devices. *Reservoir Computing: Theory, Physical Implementations, and Applications* (2021), 423–450.

[18] Joritz Galarraga, Aitor Arrieta Marcos, Shaukat Ali, Goiuria Sagardui, and Maite Arratibel. 2021. Genetic Algorithm-based Testing of Industrial Elevators under Passenger Uncertainty. In *2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 353–358. https://doi.org/10.1109/ISSREW53611.2021.00101

[19] Aitor Gartziandia, Aitor Arrieta, Jon Ayerdi, Miren Illarramendi, Aitor Agirre, Goiuria Sagardui, and Maite Arratibel. 2022. Machine learning-based test oracles for performance testing of cyber-physical systems: An industrial case study on elevators dispatching algorithms. *Journal of Software: Evolution and Process* 34, 11 (2022), e2465. https://doi.org/10.1002/smr.2465

[20] Farshad Gholami, Niousha Attar, Hassan Haghighi, Mojtaba Vahidi Asl, Meysam Valueian, and Saina Mohamadyari. 2018. A classifier-based test oracle for embedded software. In *2018 Real-Time and Embedded Systems and Technologies (RTEST)*. IEEE, 104–111. https://doi.org/10.1109/RTEST.2018.8397165

[21] Sanjib Ghosh, Tanjung Krisnanda, Tomasz Paterek, and Timothy CH Liew. 2021. Realising and compressing quantum circuits with quantum reservoir computing. *Communications Physics* 4, 1 (2021), 105.

[22] L. C. G. Govia, G. J. Ribeill, G. E. Rowlands, H. K. Krovi, and T. A. Ohki. 2021. Quantum reservoir computing with a single nonlinear oscillator. *Phys. Rev. Res.* 3 (Jan 2021), 013077. Issue 1. https://doi.org/10.1103/PhysRevResearch.3.013077

[23] Liping Han, Shaukat Ali, Tao Yue, Aitor Arrieta, and Maite Arratibel. 2023. Uncertainty-Aware Robustness Assessment of Industrial Elevator Systems. *ACM Trans. Softw. Eng. Methodol.* 32, 4, Article 95 (may 2023), 51 pages. https://doi.org/10.1145/3576041

[24] Liping Han, Tao Yue, Shaukat Ali, Aitor Arrieta, and Maite Arratibel. 2022. Are elevator software robust against uncertainties? results and experiences from an industrial case study. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (, Singapore, Singapore,) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1331–1342. https://doi.org/10.1145/3540250.3558955

[25] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. 2006. Extreme learning machine: theory and applications. *Neurocomputing* 70, 1-3 (2006), 489–501.

[26] Luca Innocenti, Salvatore Lorenzo, Ivan Palmisano, Alessandro Ferraro, Mauro Paternostro, and Gioacchino Massimo Palma. 2023. Potential and limitations of quantum extreme learning machines. *Communications Physics* 6, 1 (2023), 118.

[27] Hu Jin, Yi Wang, Nian-Wei Chen, Zhi-Jian Gou, and Shuo Wang. 2008. Artificial neural network for automatic test oracles generation. In *2008 International Conference on Computer Science and Software Engineering*, Vol. 2. IEEE, 727–730.

[28] Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M Chow, and Jay M Gambetta. 2017. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *nature* 549, 7671 (2017), 242–246.

[29] Hiroki Kawai and Yuya O Nakagawa. 2020. Predicting excited states from ground state wavefunction by supervised quantum machine learning. *Machine Learning: Science and Technology* 1, 4 (2020), 045027.

[30] Barbara Kitchenham, Lech Madeyski, David Budgen, Jacky Keung, Pearl Brereton, Stuart Charters, Shirley Gibbs, and Amnart Pohthong. 2017. Robust Statistical Methods for Empirical Software Engineering. *Empirical Softw. Engg.* 22, 2 (apr 2017), 579–630. https://doi.org/10.1007/s10664-016-9437-5

[31] Wellington Makondo, Raghava Nallanthighal, Innocent Mapanga, and Prudence Kadebu. 2016. Exploratory test oracle using multi-layer perceptron neural network. In *2016 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*. IEEE, 1166–1171.

[32] Ye Mao, Feng Boqin, Zhu Li, and Lin Yao. 2006. Neural networks based automated test oracle for software testing. In *Proceedings of the 13th International Conference on Neural Information Processing - Volume Part III* (Hong Kong, China) *(ICONIP'06)*. Springer-Verlag, Berlin, Heidelberg, 498–507.

[33] Rodrigo Martínez-Peña, Johannes Nokkala, Gian Luca Giorgi, Roberta Zambrini, and Miguel C Soriano. 2020. Information processing capacity of spin-based quantum reservoir computing systems. *Cognitive Computation* (2020), 1–12.

[34] Francesco Mezzadri. 2006. How to generate random matrices from the classical compact groups. *arXiv preprint math-ph/0609050* (2006).

[35] Amin Karimi Monsefi, Behzad Zakeri, Sanaz Samsam, and Morteza Khashehchi. 2019. Performing Software Test Oracle Based on Deep Neural Network with Fuzzy Inference System. In *High-Performance Computing and Big Data Analysis*, Lucio Grandinetti, Seyedeh Leili Mirtaheri, and Reza Shahbazian (Eds.). Springer International Publishing, Cham, 406–417.

[36] Pere Mujal, Rodrigo Martínez-Peña, Johannes Nokkala, Jorge García-Beni, Gian Luca Giorgi, Miguel C. Soriano, and Roberta Zambrini. 2021. Opportunities in Quantum Reservoir Computing and Extreme Learning Machines. *Advanced Quantum Technologies* 4, 8 (2021), 2100027. https://doi.org/10.1002/qute.202100027

[37] Kohei Nakajima, Keisuke Fujii, Makoto Negoro, Kosuke Mitarai, and Masahiro Kitagawa. 2019. Boosting computational power through spatial multiplexing in quantum reservoir computing. *Physical Review Applied* 11, 3 (2019), 034021.

[38] Johannes Nokkala, Rodrigo Martínez-Peña, Gian Luca Giorgi, Valentina Parigi, Miguel C Soriano, and Roberta Zambrini. 2021. Gaussian states of continuous-variable quantum systems provide universal and versatile reservoir computing. *Communications Physics* 4, 1 (2021), 53.

[39] Orona 2024. Orona. https://www.orona-group.com/int-en/.

[40] qreservoir 2024. qreservoir. https://owenagnel.github.io/qreservoir/qreservoir.html.

[41] Om Prakash Sangwan, Pradeep Kumar Bhatia, and Yogesh Singh. 2011. Radial basis function neural network based approach to test oracle. *SIGSOFT Softw. Eng. Notes* 36, 5 (sep 2011), 1–5. https://doi.org/10.1145/2020976.2020992

[42] André Sequeira, Luis Paulo Santos, and Luís Soares Barbosa. 2022. Variational quantum policy gradients with an application to quantum control. *arXiv e-prints* (2022), arXiv–2203.

[43] Seyed Reza Shahamiri, Wan Mohd Nasir Wan Kadir, and Suhaimi bin Ibrahim. 2010. An automated oracle approach to test decision-making structures. In *2010 3rd International Conference on Computer Science and Information Technology*, Vol. 5. IEEE, 30–34. https://doi.org/10.1109/ICCSIT.2010.5563989

[44] Seyed Reza Shahamiri, Wan Mohd Nasir Wan Kadir, Suhaimi Ibrahim, and Siti Zaiton Mohd Hashim. 2011. An automated framework for software test oracle. *Inf. Softw. Technol.* 53, 7 (jul 2011), 774–788. https://doi.org/10.1016/j.infsof.2011.02.006

[45] Seyed Reza Shahamiri, Wan MN Wan-Kadir, Suhaimi Ibrahim, and Siti Zaiton Mohd Hashim. 2012. Artificial neural networks as multi-networks automated test oracle. *Automated Software Engineering* 19, 3 (2012), 303–334.

[46] Abhishek Singhal and Abhay Bansal. 2014. Generation of test oracles using neural network and decision tree model. In *2014 5th International Conference-Confluence The Next Generation Information Technology Summit (Confluence)*. IEEE, 313–318. https://doi.org/10.1109/CONFLUENCE.2014.6949311

[47] Abhishek Singhal, Abhay Bansal, and Avadhesh Kumar. 2016. An approach to design test oracle for aspect oriented software systems using soft computing approach. *International Journal of System Assurance Engineering and Management* 7, 1 (2016), 1–5.

[48] Andrea Stocco, Paulo J. Nunes, Marcelo D'Amorim, and Paolo Tonella. 2023. ThirdEye: Attention Maps for Safe Autonomous Driving Systems. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (, Rochester, MI, USA,) *(ASE '22)*. Association for Computing Machinery, New York, NY, USA, Article 102, 12 pages. https://doi.org/10.1145/3551349.3556968

[49] Andrea Stocco, Michael Weiss, Marco Calzana, and Paolo Tonella. 2020. Misbehaviour prediction for autonomous driving systems. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (Seoul, South Korea) *(ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 359–371. https://doi.org/10.1145/3377811.3380353

[50] Yasunari Suzuki, Yoshiaki Kawase, Yuya Masumura, Yuria Hiraga, Masahiro Nakadai, Jiabao Chen, Ken M Nakanishi, Kosuke Mitarai, Ryosuke Imai, Shiro Tamiya, et al. 2021. Qulacs: a fast and versatile quantum circuit simulator for research purpose. *Quantum* 5 (2021), 559.

[51] Gouhei Tanaka, Toshiyuki Yamane, Jean Benoit Héroux, Ryosho Nakane, Naoki Kanazawa, Seiji Takeda, Hidetoshi Numata, Daiju Nakano, and Akira Hirose. 2019. Recent advances in physical reservoir computing: A review. *Neural Networks* 115 (2019), 100–123.

[52] Pablo Valle, Aitor Arrieta, and Maite Arratibel. 2023. Applying and Extending the Delta Debugging Algorithm for Elevator Dispatching Algorithms (Experience Paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (, Seattle, WA, USA,) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1055–1067. https://doi.org/10.1145/3597926.3598117

[53] Pablo Valle, Aitor Arrieta, and Maite Arratibel. 2023. Automated Misconfiguration Repair of Configurable Cyber-Physical Systems with Search: An Industrial Case Study on Elevator Dispatching Algorithms. In *Proceedings of the 45th International Conference on Software Engineering: Software Engineering in Practice* (Melbourne, Australia) *(ICSE-SEIP '23)*. IEEE Press, 396–408. https://doi.org/10.1109/ICSE-SEIP58684.2023.00042

[54] Meenakshi Vanmali, Mark Last, and Abraham Kandel. 2002. Using a neural network in the software testing process. *International Journal of Intelligent Systems* 17, 1 (2002), 45–62.

[55] Wei Xia, Jie Zou, Xingze Qiu, Feng Chen, Bing Zhu, Chunhe Li, Dong-Ling Deng, and Xiaopeng Li. 2023. Configured quantum reservoir computing for multi-task machine learning. *Science Bulletin* 68, 20 (2023), 2321–2329. https://doi.org/10.1016/j.scib.2023.08.040

[56] Qinghua Xu, Shaukat Ali, Tao Yue, and Maite Arratibel. 2022. Uncertainty-aware transfer learning to evolve digital twins for industrial elevators. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1257–1268. https://doi.org/10.1145/3540250.3558957

[57] Ran Zhang, Ya-wen Wang, and Ming-zhe Zhang. 2019. Automatic Test Oracle Based on Probabilistic Neural Networks. In *Recent Developments in Intelligent Computing, Communication and Devices*, Srikanta Patnaik and Vipul Jain (Eds.). Springer Singapore, Singapore, 437–445.

[58] Jianjun Zhao. 2020. Quantum Software Engineering: Landscapes and Horizons. *CoRR* abs/2007.07047 (2020). arXiv:2007.07047 https://arxiv.org/abs/2007.07047

Received 2024-02-08; accepted 2024-04-18